COMPARAÇÃO ENTRE PROCESSAMENTO SEQUENCIAL E PARALELO COM PYTHON

Integrantes: Marjory Letícia e Emerson Sousa

Disciplina: Fundamentos de Computação Concorrente,paralela e Distribuída

Instituição: IFPE- Palmares

Data: Outubro de 2026

1. INTRODUÇÃO

O processamento paralelo é uma técnica que permite dividir uma tarefa em partes menores para que sejam executadas simultaneamente por diferentes processos. Essa abordagem pode reduzir o tempo necessário para realizar determinadas atividades computacionais.

Neste projeto, foi desenvolvido um programa em Python para comparar o desempenho do processamento sequencial com o processamento paralelo na contagem de números primos.

2. OBJETIVO

O objetivo do projeto é analisar a diferença de desempenho entre o processamento sequencial e o paralelo, utilizando diferentes quantidades de processos para realizar a mesma tarefa.

Para isso, o programa identifica e contabiliza os números primos existentes no intervalo de 2 até 2.000.000, registrando o tempo de execução de cada configuração.

3. TECNOLOGIAS UTILIZADAS

Foram utilizadas as seguintes tecnologias e bibliotecas:

- **Python 3.8.10:** linguagem utilizada no desenvolvimento do programa.
- **multiprocessing:** biblioteca utilizada para executar tarefas em processos paralelos.
- **time:** biblioteca utilizada para medir o tempo de execução.
- **statistics:** biblioteca utilizada para calcular a média dos tempos registrados.
- **PowerShell:** terminal utilizado para executar o programa no computador.

4. DESENVOLVIMENTO

O programa foi desenvolvido para identificar números primos e comparar duas formas de processamento.

No processamento sequencial, todos os números do intervalo são analisados em uma única sequência de execução.

No processamento paralelo, o intervalo é dividido em partes menores, distribuídas entre diferentes processos. Ao final, os resultados parciais são somados para obter a quantidade total de números primos.

Foram realizadas três execuções para cada configuração, utilizando o tempo médio como referência para a comparação. O experimento foi executado com um limite de 2.000.000.

As configurações avaliadas foram:

- Processamento sequencial;
- Processamento paralelo com 1 processo;
- Processamento paralelo com 2 processos;
- Processamento paralelo com 4 processos.

5. RESULTADOS E ANÁLISE

Os testes apresentaram os seguintes resultados:

Configuração	Tempo médio
Sequencial	7,087 segundos
Paralelo com 1 processo	6,954 segundos
Paralelo com 2 processos	4,517 segundos
Paralelo com 4 processos	3,795 segundos

Todas as configurações encontraram corretamente **148.933 números primos**, demonstrando que os resultados obtidos foram consistentes entre as diferentes formas de execução.

A configuração com um processo apresentou um tempo semelhante ao processamento sequencial. Já a utilização de dois processos reduziu o tempo médio para 4,517 segundos. O melhor resultado foi obtido com quatro processos, que concluíram a tarefa em 3,795 segundos, contra 7,087 segundos do processamento sequencial.

A aceleração obtida com quatro processos foi de aproximadamente **1,87 vezes** em relação ao processamento sequencial. Isso representa uma redução de aproximadamente 46,5% no tempo de execução.

Os resultados demonstram que, para esse experimento, a divisão do trabalho entre múltiplos processos melhorou o desempenho. Entretanto, o ganho depende das características da tarefa e dos recursos disponíveis no computador.

6. EVIDÊNCIAS DO EXPERIMENTO

A captura de tela com os resultados obtidos durante a execução do programa está armazenada na pasta `evidencias`, localizada no diretório do projeto no GitHub. O arquivo apresenta os tempos médios de execução, a quantidade de números primos encontrados e os valores de aceleração calculados.

7. COMO EXECUTAR O PROGRAMA

Para executar o projeto, é necessário ter o Python instalado no computador.

1. Baixe ou clone o repositório do projeto.
2. Abra o terminal na pasta em que o arquivo `processamento_paralelo.py` está localizado.
3. Execute o seguinte comando:
   `py processamento_paralelo.py`
4. Aguarde a conclusão dos testes.
5. Confira no terminal os tempos médios e a comparação entre as configurações.

O programa realiza automaticamente as execuções e apresenta o resumo dos resultados ao final.

8. REPOSITÓRIO DO PROJETO

O código-fonte e a captura de tela dos resultados estão disponíveis no GitHub, permitindo consultar os arquivos utilizados no desenvolvimento do experimento.

Link do repositório: https://github.com/emersonsousa0/processamento-paralelo-python.git

9. CONCLUSÃO

O projeto permitiu observar, na prática, as diferenças entre o processamento sequencial e o processamento paralelo utilizando Python.

Nos testes realizados, a configuração com quatro processos apresentou o melhor desempenho, reduzindo o tempo médio de execução de 7,087 para 3,795 segundos. A quantidade de números primos encontrada permaneceu igual em todas as configurações.

Conclui-se que o processamento paralelo pode melhorar o desempenho de tarefas computacionais que podem ser divididas em partes independentes. O experimento também demonstrou a importância de medir e comparar o tempo de execução para avaliar os benefícios dessa técnica.

REFERÊNCIAS

PYTHON SOFTWARE FOUNDATION. *Python 3 Documentation: multiprocessing — Process-based parallelism*. Disponível em: https://docs.python.org/3/library/multiprocessing.html. Acesso em: 19 out. 2026.

PYTHON SOFTWARE FOUNDATION. *Python 3 Documentation: time — Time access and conversions*. Disponível em: https://docs.python.org/3/library/time.html. Acesso em: 19 out. 2026.